Assignment 08

Published | Assign To | Edit | ⋮

Learning Objectives

The goal of this assignment is to learn about DNS, email, and set up the web application to auto-scale up as demand increases and scale down when demand/load decreases. Because of the auto-scaler's dynamic nature, the lifecycle of instances is now auto-managed. To provide a static endpoint for our users, we must put a load balancer in front of the web application. The load balancer will track active instances and route traffic to them based on our defined policy.

Domain Name

Namecheap offers a free domain with the GitHub Student Developer pack, but only `.me` *TLD domain.*

1. Register a domain name with a domain registrar such as **Namecheap (https://www.namecheap.com/)** , **AWS (https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-register.html)** , **gandi.net (https://www.gandi.net/en-US)** , or **GoDaddy (https://www.godaddy.com/)** .
2. Your domain name can be anything.

DNS Setup - Configuring Amazon Route 53

This setup should be done manually from the AWS console.

By default, your new domain will be set up to use the registrar's name servers.

Reference Documentation:

- **AWS Creating Hosted Zone (http://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CreatingHostedZone.html)**
- **AWS Creating New Subdomain (https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CreatingNewSubdomain.html)**
- **How to Change DNS For a Domain in Namecheap (https://www.namecheap.com/support/knowledgebase/article.aspx/767/10/how-can-i-change-the-nameservers-for-my-domain)**

You will need to make the following changes.

Create A Hosted Zone For The Domain In The Root AWS Account

1. Create a public hosted zone in Amazon Route 53 for the `domainyourdomainname.tld` .
2. Configure Namecheap to use custom name servers provided by Amazon Route 53, which will utilize the Route 53 name servers.

Create A Subdomain & Hosted Zone For The Dev AWS Account

1. Create a public hosted zone in Amazon Route 53 for the subdomain `dev.yourdomainname.tld`.
2. Configure name servers for the subdomain in the root account. See docs.

Create A Subdomain & Hosted Zone For The Demo AWS Account

1. Create a public hosted zone in Amazon Route 53 for the subdomain `demo.yourdomainname.tld`.
2. Configure name servers for the subdomain in the root account. See docs.

Infrastructure as Code w/Terraform

IAM Roles & Policies Updates

Warning

- The assignment objective requires you to identify the necessary permissions and create corresponding roles and policies.
- Not all required roles and policies are listed in this assignment or on the lecture page.

Create Load Balancer Security Group

- Create a security group for the load balancer to access the web application.
- Add `ingress` rule to allow TCP traffic on ports `80`, and `443` from anywhere in the world.
- This security group will be referred to as the `load balancer` security group.

Update App Security Group

- Update the EC2 security group for your EC2 instances that will host web applications.
- The `ingress` rule should allow TCP traffic on ports `22` and port on which your application runs.
- The `source` of the traffic should be the `load balancer` security group.
- `Restrict` access to the instance from the `internet`.

The Auto Scaling Application Stack

So far, our web application has been accessible via the HTTP protocol from the IP address of the EC2 instance. We will now disable direct access to our web application using the IP address of the EC2 instance. The web application will now only be accessible from the load balancer.

Set up Autoscaling for EC2 Instances

Instead of launching standalone EC2 instances, we are now going to launch them in an auto-scaling group with a `minimum` of *3* instances and a `maximum` of ***5***. Use the default (or customize them as per your

preference) for properties not listed below.

Launch Templates

Key	Value
AMI	Your custom AMI
Instance Type	ANY
KeyName	YOUR_AWS_KEYNAME
AssociatePublicIpAddress	True
UserData	SAME_USER_DATA_AS_CURRENT_EC2_INSTANCE
IAM Role	SAME_AS_CURRENT_EC2_INSTANCE
Security Group	WebAppSecurityGroup

Auto Scaling Group

You should add tags AutoScalingGroup TagProperty (https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-as-tags.html) to the EC2 instances in your Auto Scaling Group.

Parameter	Value
Cooldown	60
Launch Template Name	csye6225_asg
MinSize	3
MaxSize	5
DesiredCapacity	1

AutoScaling Policies

Create the following autoscaling policies:

- Scale `up` policy when `average CPU usage` is above `5%`. Increment by `1`.
- Scale `down` policy when `average CPU usage` is below `3%`. Decrement by `1`.

Set up Application Load Balancer For Your Web Application

- EC2 instances launched in the auto-scaling group should now be load-balanced.
- Add a balancer resource to your Terraform IaC.
- Set up an Application Load Balancer to accept `HTTP` traffic on the port `80` and forward it to your application instances on the port they listen on. **You are not required to support HTTP to HTTPS redirection.**
- Attach the load balancer security group to the load balancer.

DNS Updates

- Route53 should be updated from the Terraform IaC.
- Route53 resource record for your domain name should now be an alias for your load balancer application.
- `Add` or `Update` `A` record to the Route53 zone so that your domain points to your load balancer and your web application is accessible at `http://(dev|demo).your-domain-name.tld/`.
- Your application must be accessible using the root context, i.e. `http://(dev|demo).your-domain-name.tld:<port>/` and not `http://(dev|demo).your-domain-name.tld/app-0.1/`.

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures

- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Grading Guidelines

Warning: The following guidelines are for information only and are subject to change at the discretion of the instructor and TA.

Common Grading Rubrics

Git

1. All commits to GitHub repositories must be made via pull request. No changes can be made by pushing commits directly to the main branch. The exception to this is the first commit made to the repository to initialize it, which can be done directly from the UI.

2. All pull requests must have status checks and be enforced. Merging pull requests that did not pass status checks may result in penalties.

Security

1. Committing any kind of credentials to the code or uploading them as part of your Canvas submission may result in a penalty.
2. Service providers may automatically disable your keys or suspend your account if they detect keys being committed in your GitHub repositories, even if they are private. This can significantly impact your assignment grade and your ability to complete assignments in the course. This is not grounds for assignment extensions; late penalties will apply as documented in the syllabus.

Software Engineering Best Practices

1. Values for variables should not be hardcoded in the code. It's OK to set default values for variables, but if the value cannot be modified during the demo or when the TA or instructor asks, you may be penalized.
2. Follow industry standards and use a style guide for your code. You can find them with simple web searches.

Code Stability, Error Handling, & Edge Cases

1. The application should not crash between various API calls.
2. The application, VM, or other resources should not require a restart during the demo unless the grading guidelines require it to demonstrate a specific feature/functionality.
3. The server cannot respond with a 500 Internal Server Error. This usually highlights areas of code that do not have proper error handling.

Previous Assignment Objectives

TAs must verify that students have resolved all open issues from the previous assignment(s).

Git & GitHub Repository (2%)

Git Repository Content Check

1. Check the repository for any IDE-specific files. IDE configuration files must not be in the repository.
2. Verify their .gitignore configuration.
3. Check the repository for build artifacts such as `.class`, `.jar`, `.war` files and `build`, `node_modules`, `venv` directory. None of these should be checked into the repository.
4. Check for dependencies. Dependencies from the Maven repository, npm, or Python should not be committed to the git repository.

Git Forking Workflow

1. No direct commits are made to the organization's repository. Verify that the student works from a forked repository and uses the feature branch. There should be no direct commits to the main branch

in their forked repository.

DNS

Domain Name & Zone Delegation for Sub-Domains

1. The student has registered a domain name or has an existing domain name to use in the course.
2. `dev.domain.tld` zone is delegated to Route53 in the `DEV` AWS account.
3. `demo.domain.tld` zone is delegated to Route53 in the `DEMO` AWS account.

Emails

1. The student has a service set up to send emails for their custom domain.
2. Check for DKIM and SPF records using the `dig` command.

Security Groups

- A new security group is created and attached to the load balancer with an ingress rule to allow TCP traffic on ports 80 and 443 from anywhere in the world.
- The `application security group` no longer allows direct access to the web application from the internet. Only traffic on the port `22` is allowed. The ingress rule allows traffic from the `load balancer security group` for the web application.

Autoscaler

- The EC2 instance is now launched as part of an auto-scaling group with a minimum of `3` instance and a maximum of 5 or higher.
- Students to demo putting the load on the web application, and an auto-scaling event being triggered to launch more instances.
- Students to demo reducing load on the web application and auto-scaling event being triggered to terminate some of the running instances.
- The web application is automatically started and configured when launched by the auto-scaler.

Load Balancer

- EC2 instances launched in the auto-scaling group should now be load-balanced.
- The Application load balancer listens for HTTP traffic on port 80 and forwards it to the application instances on whatever port it listens on. Students are not required to support HTTP to HTTPS redirection.

- The load balancer security group is attached to the load balancer.

DNS

- Type A record is added/updated to the Route53 zone so web application accessible at http://(dev|demo).your-domain-name.tld

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Nov 6 at 3pm	Everyone	Oct 30 at 6pm	Nov 7 at 3pm

+ **Rubric**